Complete Attribute Grammar with Production and SDT Rules

Global Variables and Data Structures

```c
// Global variables for SDT processing
int flag = 0;          // 0: symbol table population, 1: label setup, 2: code generation
int offset = 0;        // current offset for variable allocation
int temp_count = 0;    // counter for temporary variables
int label_count = 0;   // counter for labels
SymbolTable* symtab;   // global symbol table
ScopeStack* scope_stack; // stack for nested scopes
FunctionTable* func_table; // table for function declarations
int current_scope = 0;  // current scope level
```

Production Rules with SDT Rules

1. Program Structure

```
P → F P1
{
   P.code = F.code || P1.code
}

P → D S
{
   P.code = D.code || S.code
   P.next = newlabel()
}

P → S
{
   P.code = S.code
   P.next = newlabel()
}
```

1.1. Function Declaration Rules

```
F → T id ( PL ) { D S }
{
   if (flag == 0) {
      enter_function(id.lexeme, T.type, PL.params)
      push_scope()
```

```
        // Enter parameters into symbol table
        enter_params(PL.params)
    }
    if (flag == 2) {
        F.code = gen("proc " id.lexeme) ||
                 D.code ||
                 S.code ||
                 gen("endproc")
    } else {
        F.code = ""
    }
    if (flag == 0) {
        pop_scope()
    }
}

F → T id ( ) { D S }
{
    if (flag == 0) {
        enter_function(id.lexeme, T.type, null)
        push_scope()
    }
    if (flag == 2) {
        F.code = gen("proc " id.lexeme) ||
                 D.code ||
                 S.code ||
                 gen("endproc")
    } else {
        F.code = ""
    }
    if (flag == 0) {
        pop_scope()
    }
}

PL → PL1 , P
{
    PL.params = PL1.params ++ P.param
}

PL → P
{
    PL.params = [P.param]
}

P → T id
{
```

```
    P.param = {type: T.type, name: id.lexeme, width: T.width}
}
```

2. Declaration Rules

```
D → D₁ V
{
    D.code = D₁.code
    if (flag == 0) {
        // Symbol table population phase
        enter_symbol(V.id, V.type, V.width, offset)
        offset = offset + V.width
    }
}

D → V
{
    D.code = ""
    if (flag == 0) {
        enter_symbol(V.id, V.type, V.width, offset)
        offset = offset + V.width
    }
}

V → T id ;
{
    V.id = id.lexeme
    V.type = T.type
    V.width = T.width
}

V → T id [ num₁ ] [ num₂ ] ;
{
    V.id = id.lexeme
    V.type = array(num₂.val, array(num₁.val, T.type))
    V.width = num₁.val * num₂.val * T.width
}
```

3. Type Rules

```
T → int
{
    T.type = integer
    T.width = 4
}
```

```
T → float
{
   T.type = float
   T.width = 8
}

T → char
{
   T.type = character
   T.width = 1
}
```

4. Statement Rules

```
S → S1 S2
{
   S1.next = S2.begin
   S.code = S1.code || S2.code
   S.begin = S1.begin
}

S → A
{
   S.code = A.code
   S.begin = newlabel()
}

S → W
{
   S.code = W.code
   S.begin = W.begin
}

S → F_LOOP
{
   S.code = F_LOOP.code
   S.begin = F_LOOP.begin
}

S → I
{
   S.code = I.code
   S.begin = I.begin
}
```

```
S → CALL
{
   S.code = CALL.code
   S.begin = newlabel()
}

S → { D S1 }
{
   if (flag == 0) {
      push_scope()
   }
   S.code = D.code || S1.code
   S.begin = S1.begin
   if (flag == 0) {
      pop_scope()
   }
}

S → { S1 }
{
   S.code = S1.code
   S.begin = S1.begin
}
```

5. Assignment Statement Rules

```
A → id = E ;
{
   if (flag == 2) {
      A.code = E.code || gen(id.place " = " E.place)
   } else {
      A.code = ""
   }
}

A → id [ E1 ] [ E2 ] = E3 ;
{
   if (flag == 2) {
      t1 = newtemp()
      t2 = newtemp()
      t3 = newtemp()
      A.code = E1.code || E2.code || E3.code ||
               gen(t1 " = " E1.place " * " width(lookup(id))) ||
               gen(t2 " = " E2.place " * " base_width(lookup(id))) ||
               gen(t3 " = " t1 " + " t2) ||
               gen(id.place "[" t3 "]" " = " E3.place)
```

```
    } else {
        A.code = ""
    }
}
```

6. For Loop Rules

```
F_LOOP → for ( A1 ; B ; A2 ) S1
{
    F_LOOP.begin = newlabel()
    test_label = newlabel()
    update_label = newlabel()
    B.true = S1.begin
    B.false = F_LOOP.next
    S1.next = update_label

    if (flag == 1) {
        // Label setup phase
        F_LOOP.code = ""
    } else if (flag == 2) {
        // Code generation phase
        F_LOOP.code = A1.code ||
                      gen("label " test_label) ||
                      B.code ||
                      gen("label " S1.begin) ||
                      S1.code ||
                      gen("label " update_label) ||
                      A2.code ||
                      gen("goto " test_label) ||
                      gen("label " F_LOOP.next)
    } else {
        F_LOOP.code = ""
    }
}

F_LOOP → for ( A1 ; B ; A2 ) { D S1 }
{
    if (flag == 0) {
        push_scope()
    }

    F_LOOP.begin = newlabel()
    test_label = newlabel()
    update_label = newlabel()
    B.true = S1.begin
    B.false = F_LOOP.next
```

```
    S1.next = update_label

    if (flag == 1) {
        F_LOOP.code = ""
    } else if (flag == 2) {
        F_LOOP.code = A1.code ||
                      gen("label " test_label) ||
                      B.code ||
                      gen("label " S1.begin) ||
                      D.code ||
                      S1.code ||
                      gen("label " update_label) ||
                      A2.code ||
                      gen("goto " test_label) ||
                      gen("label " F_LOOP.next)
    } else {
        F_LOOP.code = D.code
    }

    if (flag == 0) {
        pop_scope()
    }
}
```

```
W → while ( B ) S1
{
   W.begin = newlabel()
   B.true = S1.begin
   B.false = W.next
   S1.next = W.begin

   if (flag == 1) {
      // Label setup phase
      W.code = ""
   } else if (flag == 2) {
      // Code generation phase
      W.code = gen("label " W.begin) ||
               B.code ||
               gen("label " S1.begin) ||
               S1.code ||
               gen("goto " W.begin) ||
               gen("label " W.next)
   } else {
      W.code = ""
   }
}
```

8. If-Else Statement Rules (with Local Declarations)

```
I → if ( B ) S1 else S2
{
   B.true = S1.begin
   B.false = S2.begin
   S1.next = I.next
   S2.next = I.next

   if (flag == 1) {
      // Label setup phase
      I.code = ""
      I.begin = newlabel()
   } else if (flag == 2) {
      // Code generation phase
      I.code = B.code ||
               gen("label " S1.begin) ||
               S1.code ||
               gen("goto " I.next) ||
               gen("label " S2.begin) ||
               S2.code ||
               gen("label " I.next)
```

```
    } else {
        I.code = ""
    }
}

I → if ( B ) S1
{
    B.true = S1.begin
    B.false = I.next
    S1.next = I.next

    if (flag == 1) {
        I.code = ""
        I.begin = newlabel()
    } else if (flag == 2) {
        I.code = B.code ||
                 gen("label " S1.begin) ||
                 S1.code ||
                 gen("label " I.next)
    } else {
        I.code = ""
    }
}

I → if ( B ) { D1 S1 } else { D2 S2 }
{
    if (flag == 0) {
        push_scope()
        // Process D1 in if-branch scope
        pop_scope()
        push_scope()
        // Process D2 in else-branch scope
        pop_scope()
    }

    B.true = S1.begin
    B.false = S2.begin
    S1.next = I.next
    S2.next = I.next

    if (flag == 1) {
        I.code = ""
        I.begin = newlabel()
    } else if (flag == 2) {
        I.code = B.code ||
                 gen("label " S1.begin) ||
                 D1.code ||
```

```
            S1.code ||
            gen("goto " I.next) ||
            gen("label " S2.begin) ||
            D2.code ||
            S2.code ||
            gen("label " I.next)
    } else {
        I.code = D1.code || D2.code
    }
}

I → if ( B ) { D S1 }
{
    if (flag == 0) {
        push_scope()
    }

    B.true = S1.begin
    B.false = I.next
    S1.next = I.next

    if (flag == 1) {
        I.code = ""
        I.begin = newlabel()
    } else if (flag == 2) {
        I.code = B.code ||
                gen("label " S1.begin) ||
                D.code ||
                S1.code ||
                gen("label " I.next)
    } else {
        I.code = D.code
    }

    if (flag == 0) {
        pop_scope()
    }
}
```

9. Function Call Rules

```
CALL → id ( AL ) ;
{
    if (flag == 2) {
        CALL.code = AL.code ||
                gen("call " id.lexeme ", " AL.arg_count)
```

```
    } else {
        CALL.code = ""
    }
}

CALL → id ( ) ;
{
    if (flag == 2) {
        CALL.code = gen("call " id.lexeme ", 0")
    } else {
        CALL.code = ""
    }
}

E → id ( AL )
{
    if (flag == 2) {
        E.place = newtemp()
        E.code = AL.code ||
                 gen("call " id.lexeme ", " AL.arg_count) ||
                 gen(E.place " = " "retval")
    } else {
        E.place = ""
        E.code = ""
    }
    E.type = lookup_function(id.lexeme).return_type
}

E → id ( )
{
    if (flag == 2) {
        E.place = newtemp()
        E.code = gen("call " id.lexeme ", 0") ||
                 gen(E.place " = " "retval")
    } else {
        E.place = ""
        E.code = ""
    }
    E.type = lookup_function(id.lexeme).return_type
}

AL → AL1 , E
{
    if (flag == 2) {
        AL.code = AL1.code || E.code ||
                  gen("param " E.place)
        AL.arg_count = AL1.arg_count + 1
```

```
    } else {
        AL.code = ""
        AL.arg_count = AL1.arg_count + 1
    }
}

AL → E
{
    if (flag == 2) {
        AL.code = E.code ||
                gen("param " E.place)
        AL.arg_count = 1
    } else {
        AL.code = ""
        AL.arg_count = 1
    }
}
```

```
B → B1 || B2
{
   B1.true = B.true
   B1.false = newlabel()
   B2.true = B.true
   B2.false = B.false

   if (flag == 2) {
      B.code = B1.code ||
               gen("label " B1.false) ||
               B2.code
   } else {
      B.code = ""
   }
}

B → B1 && B2
{
   B1.true = newlabel()
   B1.false = B.false
   B2.true = B.true
   B2.false = B.false

   if (flag == 2) {
      B.code = B1.code ||
               gen("label " B1.true) ||
               B2.code
   } else {
      B.code = ""
   }
}

B → ! B1
{
   B1.true = B.false
   B1.false = B.true
   B.code = B1.code
}

B → ( B1 )
{
   B1.true = B.true
   B1.false = B.false
   B.code = B1.code
}
```

```
B → E1 relop E2
{
   if (flag == 2) {
      B.code = E1.code || E2.code ||
            gen("if " E1.place " " relop.op " " E2.place " goto " B.true) ||
            gen("goto " B.false)
   } else {
      B.code = ""
   }
}

B → true
{
   if (flag == 2) {
      B.code = gen("goto " B.true)
   } else {
      B.code = ""
   }
}

B → false
{
   if (flag == 2) {
      B.code = gen("goto " B.false)
   } else {
      B.code = ""
   }
}
```

9. Arithmetic Expression Rules

```
E → E1 + E2
{
   if (flag == 2) {
      E.place = newtemp()
      E.code = E1.code || E2.code ||
            gen(E.place " = " E1.place " + " E2.place)
   } else {
      E.place = ""
      E.code = ""
   }
   E.type = arith_type(E1.type, E2.type)
}

E → E1 - E2
{
```

```
    if (flag == 2) {
        E.place = newtemp()
        E.code = E1.code || E2.code ||
                 gen(E.place " = " E1.place " - " E2.place)
    } else {
        E.place = ""
        E.code = ""
    }
    E.type = arith_type(E1.type, E2.type)
}

E → E1 * E2
{
    if (flag == 2) {
        E.place = newtemp()
        E.code = E1.code || E2.code ||
                 gen(E.place " = " E1.place " * " E2.place)
    } else {
        E.place = ""
        E.code = ""
    }
    E.type = arith_type(E1.type, E2.type)
}

E → E1 / E2
{
    if (flag == 2) {
        E.place = newtemp()
        E.code = E1.code || E2.code ||
                 gen(E.place " = " E1.place " / " E2.place)
    } else {
        E.place = ""
        E.code = ""
    }
    E.type = arith_type(E1.type, E2.type)
}

E → - E1
{
    if (flag == 2) {
        E.place = newtemp()
        E.code = E1.code ||
                 gen(E.place " = " "minus " E1.place)
    } else {
        E.place = ""
        E.code = ""
    }
```

```
    E.type = E1.type
}

E → ( E1 )
{
    E.place = E1.place
    E.code = E1.code
    E.type = E1.type
}
```

10. Variable and Array Access Rules

```
E → id
{
    if (flag == 2) {
        E.place = lookup(id.lexeme).place
        E.code = ""
    } else {
        E.place = ""
        E.code = ""
    }
    E.type = lookup(id.lexeme).type
}

E → id [ E1 ] [ E2 ]
{
    if (flag == 2) {
        t1 = newtemp()
        t2 = newtemp()
        t3 = newtemp()
        E.place = newtemp()
        E.code = E1.code || E2.code ||
                 gen(t1 " = " E1.place " * " width(base_type(lookup(id)))) ||
                 gen(t2 " = " E2.place " * " base_width(lookup(id))) ||
                 gen(t3 " = " t1 " + " t2) ||
                 gen(E.place " = " id.place "[" t3 "]")
    } else {
        E.place = ""
        E.code = ""
    }
    E.type = base_type(lookup(id.lexeme).type)
}

E → num
{
    if (flag == 2) {
```

```
        E.place = num.val
        E.code = ""
    } else {
        E.place = ""
        E.code = ""
    }
    E.type = integer  // or float based on num type
}
```

11. Relational Operator Rules

```
relop → <
{ relop.op = "<" }

relop → <=
{ relop.op = "<=" }

relop → >
{ relop.op = ">" }

relop → >=
{ relop.op = ">=" }

relop → ==
{ relop.op = "==" }

relop → !=
{ relop.op = "!=" }
```

Supporting Functions

Symbol Table Functions

```
c
```

```
// Scope management for nested declarations
void push_scope() {
    current_scope++;
    push(scope_stack, offset);
    // Create new scope in symbol table
}

void pop_scope() {
    // Remove symbols from current scope
    remove_scope_symbols(symtab, current_scope);
    offset = pop(scope_stack);
    current_scope--;
}

// Enter symbol into symbol table with scope
void enter_symbol(char* name, Type type, int width, int offset) {
    SymbolEntry* entry = malloc(sizeof(SymbolEntry));
    entry->name = name;
    entry->type = type;
    entry->width = width;
    entry->offset = offset;
    entry->scope = current_scope;
    entry->place = name;  // for simple variables
    insert_symbol(symtab, entry);
}

// Enter function into function table
void enter_function(char* name, Type return_type, ParamList* params) {
    FunctionEntry* entry = malloc(sizeof(FunctionEntry));
    entry->name = name;
    entry->return_type = return_type;
    entry->params = params;
    entry->param_count = count_params(params);
    insert_function(func_table, entry);
}

// Enter parameters into symbol table
void enter_params(ParamList* params) {
    ParamList* current = params;
    while (current != NULL) {
        enter_symbol(current->param.name, current->param.type,
                     current->param.width, offset);
        offset += current->param.width;
        current = current->next;
    }
}
```

```
// Lookup symbol in symbol table (searches all accessible scopes)
SymbolEntry* lookup(char* name) {
    return find_symbol_in_scope(symtab, name, current_scope);
}

// Lookup function in function table
FunctionEntry* lookup_function(char* name) {
    return find_function(func_table, name);
}

// Get width of a type
int width(Type t) {
    if (t == integer) return 4;
    if (t == float) return 8;
    if (t == character) return 1;
    if (t.kind == array) return t.size * width(t.element_type);
    return 0;
}

// Get base width for array calculations
int base_width(SymbolEntry* entry) {
    Type t = entry->type;
    if (t.kind == array && t.element_type.kind == array) {
        return width(t.element_type.element_type);
    }
    return width(t.element_type);
}

// Get base type of array
Type base_type(Type t) {
    if (t.kind == array && t.element_type.kind == array) {
        return t.element_type.element_type;
    }
    if (t.kind == array) {
        return t.element_type;
    }
    return t;
}
```

Code Generation Functions

c

```
// Generate new temporary variable
char* newtemp() {
    char* temp = malloc(10);
    sprintf(temp, "t%d", temp_count++);
    return temp;
}

// Generate new label
char* newlabel() {
    char* label = malloc(10);
    sprintf(label, "L%d", label_count++);
    return label;
}

// Generate three-address code
char* gen(char* instruction) {
    return instruction;  // In actual implementation, this would emit to output
}

// Determine arithmetic result type
Type arith_type(Type t1, Type t2) {
    if (t1 == float || t2 == float) return float;
    if (t1 == integer && t2 == integer) return integer;
    return t1;  // default case
}
```

Complete Grammar Summary

```
P → F P
  | D S
  | S

F → T id ( PL ) { D S }
  | T id ( ) { D S }

PL → PL , P
   | P

P → T id

D → D V
  | V

V → T id ;
  | T id [ num ] [ num ] ;
```

```
T → int | float | char

S → S S
  | A
  | W
  | F_LOOP
  | I
  | CALL
  | { D S }
  | { S }

A → id = E ;
  | id [ E ] [ E ] = E ;

W → while ( B ) S
  | while ( B ) { D S }

F_LOOP → for ( A ; B ; A ) S
       | for ( A ; B ; A ) { D S }

I → if ( B ) S else S
  | if ( B ) S
  | if ( B ) { D S } else { D S }
  | if ( B ) { D S }

CALL → id ( AL ) ;
     | id ( ) ;

AL → AL , E
   | E

B → B || B
  | B && B
  | ! B
  | ( B )
  | E relop E
  | true
  | false

E → E + E
  | E - E
  | E * E
  | E / E
  | - E
  | ( E )
  | id
```

```
  | id [ E ] [ E ]
  | id ( AL )
  | id ( )
  | num

relop → < | <= | > | >= | == | !=
```

Attribute Definitions

Inherited Attributes:

- **B.true**: Label to jump to when boolean expression is true
- **B.false**: Label to jump to when boolean expression is false
- **S.next**: Label of the next statement to execute after S

Synthesized Attributes:

- **E.place**: Location (variable/temporary) holding the value of expression E
- **E.code**: Three-address code for evaluating expression E
- **E.type**: Type of expression E
- **B.code**: Three-address code for evaluating boolean expression B
- **S.code**: Three-address code for statement S
- **S.begin**: Label marking the beginning of statement S
- **V.id**: Identifier name for variable declaration
- **V.type**: Type of declared variable
- **V.width**: Storage width of declared variable
- **T.type**: Type represented by type specifier T
- **T.width**: Width of type T
- **F.code**: Three-address code for function definition
- **PL.params**: List of parameters for function
- **AL.code**: Three-address code for argument list evaluation
- **AL.arg_count**: Number of arguments in argument list
- **CALL.code**: Three-address code for function call

Bonus Features Implementation

1. Variable Declarations in Loops and If-Else (1 point)

The grammar now supports:

- **While loops with declarations**: `while (B) { D S }`

- **If-else with declarations**: `if (B) { D S } else { D S }`
- **For loops with declarations**: `for (A; B; A) { D S }`

Key Features:

- Proper scope management with `push_scope()` and `pop_scope()`
- Local variables are removed when exiting scope
- Offset management for nested scopes

2. For Loop Support (2 points)

The grammar includes complete for loop support:

- **Standard for loop**: `for (initialization; condition; update) statement`
- **For loop with local declarations**: `for (A; B; A) { D S }`

Key Features:

- Three-part structure: initialization, test, update
- Proper label management for loop control
- Support for local variable declarations within for loop body

3. Function/Procedure Call Support (2 points)

The grammar supports:

- **Function definitions**: `T id(params) { D S }`
- **Function calls as statements**: `id(args);`
- **Function calls in expressions**: `id(args)` returning values
- **Parameter passing**: Proper parameter list handling

Key Features:

- Function table management
- Parameter list processing
- Argument evaluation and passing
- Return value handling
- Proper calling conventions with `call`, `param`, `retval` instructions

Three-Phase Processing Algorithm

1. **Phase 1 (flag = 0)**: Symbol table population
   - Traverse AST and populate symbol table with variable declarations

- Calculate offsets and widths for all variables

2. **Phase 2 (flag = 1)**: Label setup
   - Traverse AST and set up inherited attributes for control flow
   - Establish true/false labels for boolean expressions
   - Set next labels for statements
3. **Phase 3 (flag = 2)**: Code generation
   - Traverse AST and generate three-address code
   - Use previously established symbol table and labels
   - Generate final executable code sequence

This attribute grammar provides a complete framework for handling variable declarations, arithmetic expressions, assignments, control flow statements, and 2D array operations while supporting the three-phase compilation approach suggested in the assignment.